

UNIT II: Classes and objects, class declaration, creating objects, methods, constructors and constructor overloading, garbage collector, importance of static keyword and examples, this keyword, arrays, command line arguments, nested classes.

Classes and objects:

Class:

- A class is a user defined data type / non-primitive data type that contains attributes and methods that operate on data. The attributes or variables defined within a **class** are called **instance variables** and the code that operates on this data is known as **methods**.

(or)

- Class can be defined as a template / blueprint that describe the variables / methods of a particular entity.
- In Java everything is encapsulated under classes. Class is the core of Java language.
- A class in java can contain:
 - ❖ **attributes / variables / fields**
 - ❖ **methods**
 - ❖ **constructor**
 - ❖ **block**
 - ❖ **class and interface**

Rules for Java Class:

- A class can have only public or default (no modifier) access modifier.
- It can be either abstract, final or concrete (normal class).
- It must have the class keyword, and class must be followed by a legal identifier.
- It may optionally implement any number of comma-separated interfaces.
- The class attributes and methods are declared within a set of curly braces {}.
- Each **.java** source file may contain only one public class. A source file may contain any number of default visible classes.
- The source file name must match the public class name and it must have a **.java** suffix.
- By naming convention, class names capitalize the initial of each word.
- For example: Employee, Boss, DateUtility, PostOffice, RegularRateCalculator.
- This type of naming convention is known as Pascal naming convention.
- The other convention, the camel naming convention, capitalizes the initial of each word, except the first word.
- Methods and attributes use the camel naming convention.

Syntax:

```
class <class_name>
{
    attributes;
    methods;
}
```

Example:

```
public class Student
{
    String name;
    int id;
    public void readData()
    {
    }
    public void displayData ()
    {
    }
};
```

Object:

An object is an instance of a class. It is a reference variable that represents attributes as well as methods required for operating on the data.

(or)

An object is an entity that exists physically in the real world which requires some memory will be called as an object.

Ex: chair, bike, marker, pen, table, car, mobile etc.

An object has three characteristics:

- **state:** represents data (value) of an object.
- **behaviour:** represents the behaviour (functionality) of an object such as deposit, withdraw etc.
- **identity:** Object identity is typically implemented via a unique ID. The value of the ID is not visible to the external user. But, it is used internally by the JVM to identify each object uniquely.

For Example: Pen is an object. Its name is Reynolds, color is white etc. known as its state. It is used to write, so writing is its behaviour.

Creating an Object:

As mentioned previously, a class provides the blueprints for objects. So basically, an object is created from a class. In Java, the **new** key word is used to create new objects.

There are three steps when creating an object from a class:

- **Declaration:** A variable declaration with a variable name with an object type.
- **Instantiation:** The 'new' key word is used to create the object.
- **Initialization:** The 'new' keyword is followed by a call to a constructor. This call initializes the new object.
- To create object of a class **<new> Keyword** can be used.

Syntax:

```
<Class_Name> ClassObjectReference = new <Class_Name>();
```

Example:

```
Student std=new Student();
```

Here constructor of the class(*Class_Name*) will get executed and object will be created(*ClassObjectReference will hold the reference of created object in memory*).

Simple Example of Object and Class:**Ex-1: main() inside the class**

```
public class Student {  
    int id;  
    String name;  
    public static void main(String[] args) {  
        Student s1 = new Student();  
        System.out.println(s1.id);  
        System.out.println(s1.name);  
    }  
}
```

Output:

```
0  
null
```

Ex-2: main() outside the class

```
class StudentInfo {  
    int id;  
    String name;  
}  
  
class Student  
{  
    public static void main(String[] args) {  
        StudentInfo si = new StudentInfo();  
        System.out.println(si.id);  
        System.out.println(si.name);  
    }  
}
```

Output:

0

Null

Different ways to create an object in Java:

Ex-1: single Object creation

Sample s1 = new Sample();

Ex-2: Multiple Objects creation

Sample s1 = new Sample();**Sample s2 = new Sample();**

Ex-3: multiple objects creation in one line

Sample s1 = new Sample(),s2 = new Sample(1,"REC");

Ex-4:

new Sample();//anonymous object(nameless Object)

Example of an anonymous object

```
class StudentInfo {
    int id;
    String name;
}
public class Student
{
    public static void main(String[] args) {
        System.out.println(new StudentInfo().id);
        System.out.println(new StudentInfo().name);
    }
}
```

Output:

0
Null

Object initialization in Java

There are 3 ways to initialize object in Java.

1. By reference variable
2. By method
3. By constructor

Initialization through reference

Initializing an object means storing data into the object.

Ex-1.

```
class StudentInfo {
    int id;
    String name;
}

class Student
{
    public static void main(String[] args) {
        StudentInfo si = new StudentInfo();
        si.id = 100;
        si.name = "Java";
        System.out.println(si.id+" "+si.name);
    }
}
```

Output:

100 Java

Ex-2. We can also create multiple objects and store information in it through reference Variable

```
class StudentInfo {
    int id;
    String name;
}

class Student
{
    public static void main(String[] args) {
        StudentInfo s1 = new StudentInfo();
        StudentInfo s2 = new StudentInfo();
        s1.id = 1;
        s1.name = "REC";
        System.out.println(s1.id+ " "+s1.name);
        s2.id = 2;
        s2.name = "RIT";
        System.out.println(s2.id+ " "+s2.name);
    }
}
```

Output:

```
1 REC
2 RIT
```

2. Initialization through method

In this example, we are creating the two objects of StudentInfo class and initializing the value to these objects by invoking the insert() method. Here, we are displaying the state (data) of the objects by invoking the display() method.

Ex:

```
class StudentInfo {
    int id;
    String name;
    void insert(int i,String s)
    {
        id=i;
        name=s;
    }
    void display()
    {
        System.out.println(id+" "+name);
    }
}

class Student
{
    public static void main(String[] args) {
        StudentInfo s1 = new StudentInfo();
        StudentInfo s2 = new StudentInfo();
        s1.insert(1,"REC");
    }
}
```

```
        s2.insert(2,"RIT");  
        s1.display();  
        s2.display();  
    }  
}
```

Output:

```
1 REC  
2 RIT
```

3. Initialization through Constructor.**Ans: Refer Constructor Examples.****Methods in Java**

- A **method** is a block of code or collection of statements or a set of code grouped together to perform a certain task or operation.

Advantages

- Code Reusability
- Code Optimization
- Write once and use it many times

Parts of a Method

- Method Declaration (Method Prototype / Method Header)
- Method Definition
- Method Call

Method Declaration

It contains Access modifier, returntype, method name and method parameters.

Syntax:

Accessmodifier returntype methodName(parameters);

Method Definition

It contains method declaration and body.

Syntax:

```
Accessmodifier returnType nameOfMethod (parameters)  
{  
    // method body  
}
```

The syntax shown above includes:

- **Accessmodifier** – It defines the access type of the method and it is optional to use.
- **returnType** – Method may return a value.
- **nameofMethod** – This is the method name. The method signature consists of the method name and the parameter list.

- **Parameters** – The list of parameters, it is the type, order, and number of parameters of a method. These are optional, method may contain zero parameters.
- **method body** – The method body contains statements or code.

Method Call

It is used to transfer control from one place to another place during program execution.

syntax:

methodName(parameters);

Types of Method

There are two types of methods in Java:

- Predefined Methods
- User-defined Methods (Instance Methods, Static Methods)

Predefined Methods

- In Java, predefined methods are the methods that are already defined in the Java class libraries are known as predefined methods.
- It is also known as the **standard library method** or **built-in method**.
- We can directly use these methods just by calling them in the program at any point.

Ex: print(), println(), length(), equals(), compareTo(), sqrt(), etc.

Ex:

```
public class Demo
{
    public static void main(String[] args)
    {
        // using the max() method of Math class
        System.out.print("The maximum number is: " + Math.max(9,7));
    }
}
```

Output:

The maximum number is: 9

User-defined Methods

- The method is created by the user or programmer is known as a **user-defined** method. These methods are modified according to the requirement.

Based on the data flow between the calling method and called method, the methods are classified as follows..

1. Method without parameters and without return value.
2. Method without parameters and with return value.
3. Method with parameters and with return value.
4. Method with parameters and without return value.

1. Method without parameters and without return values

```
1 class Sample {
2     int a=10,b=20;
3     // create a method
4     void addNumbers() {
5         int sum = a + b;
6         System.out.println("Sum is: " + sum);
7     }
8
9     public static void main(String[] args) {
10
11
12         // create an object of Main
13         Sample obj = new Sample();
14         // calling method
15         obj.addNumbers();
16     }
17 }
```

<terminated> Sample [Java Application]
Sum is: 30

2. Method without parameters and with return value

```
1 class Sample {
2     int a=10,b=20;
3     // create a method
4     int addNumbers() {
5         int sum = a + b;
6         return sum;
7     }
8
9     public static void main(String[] args) {
10
11
12         // create an object of Main
13         Sample obj = new Sample();
14         // calling method
15         int result = obj.addNumbers();
16         System.out.println("Sum is: " + result);
17     }
18 }
```

<terminated> Sample [Java Application] C:\Users\
Sum is: 30

3. Method with parameters and with return value

```

1 import java.util.*;
2 class Sample {
3     // create a method
4     int addNumbers(int a,int b) {
5         int sum = a + b;
6         return sum;
7     }
8
9     public static void main(String[] args) {
10
11         //System.in is a standard input stream
12         Scanner sc= new Scanner(System.in);
13         System.out.print("Enter first number: ");
14         int a= sc.nextInt();
15         System.out.print("Enter second number: ");
16         int b= sc.nextInt();
17
18         // create an object of Main
19         Sample obj = new Sample();
20         // calling method
21         System.out.println("Sum is: " + obj.addNumbers(a,b));
22     }
23 }

```

```

<terminated> Sample [Java Application] C:\Use
Enter first number: 10
Enter second number: 20
Sum is: 30

```

4. Method with parameters and without return value

```

1 import java.util.*;
2 class Sample {
3     int a=10,b=20;
4     // create a method
5     void addNumbers(int a,int b) {
6         int sum = a + b;
7         System.out.println("Sum is: " + sum);
8     }
9
10    public static void main(String[] args) {
11
12        //System.in is a standard input stream
13        Scanner sc= new Scanner(System.in);
14        System.out.print("Enter first number: ");
15        int a= sc.nextInt();
16        System.out.print("Enter second number: ");
17        int b= sc.nextInt();
18
19        // create an object of Main
20        Sample obj = new Sample();
21        // calling method
22        obj.addNumbers(a,b);
23    }
24 }

```

```

<terminated> Sample [Java Application] C:\Us
Enter first number: 10
Enter second number: 20
Sum is: 30

```

Instance Methods

The method of the class is known as an **instance method**. It is a **non-static** method defined in the class. Before calling or invoking the instance method, it is necessary to create an object of its class.

Syntax:

objectname. methodname ();

Ex: Refer User defined method examples**Static Methods:**

If you apply static keyword with any method, it is known as static method.

- A static method belongs to the class rather than object of a class.
- A static method can be invoked without the need for creating an instance of a class.
- static method can access static data member and can change the value of it.
- We cannot access instance variables (non-static) and instance methods (non-static methods) inside the static method.
- It is invoked by using the classname , objectname, and through the instance method.

Ex-1:

```
class Display
{
    void display()
    {
        System.out.println("Welcome To JAVA");
        display1();    // static method calling
    }
    static void display1()
    {
        System.out.println("Hello World");
    }
}

public class ClassDemo {
    public static void main(String[] args) {
        Display d = new Display(); // Object Creation
        d.display();                // instance method calling

        Display.display1();         // static method calling

        d.display1();               // static method calling
    }
}
```

Output:

```
Welcome To JAVA
Hello World
Hello World
Hello World
```

Ex-2:

```

class Student{
    int rollno;           //instance variable
    String name;
    static String college = "REC"; //static variable

    static void change(){
        college = "RIT";
    }

    //constructor to initialize the variable
    Student(int r, String n){
        rollno = r;
        name = n;
    }
    //method to display the values
    void display (){
        System.out.println(rollno+" "+name+" "+college);
    }
}

//we can change the college of all objects by the single line of code
public class StaticDemo{
    public static void main(String args[]){

        Student.change(); // static method calling

        Student s1 = new Student(1,"JAVA");
        Student s2 = new Student(2,"C++");

        s1.display();
        s2.display();
    }
}

```

Output:

```

1 JAVA RIT
2 C++ RIT

```

Method Overloading:

- Two or more methods can have the same name inside the same class if they accept different signature. This feature is known as method overloading.
- Method overloading is achieved by either:
 - changing the number of arguments.
 - or changing the data type of arguments.
 - or changing the order of arguments.
- It is not method overloading if we only change the return type of methods. There must be differences in the number of parameters.

Ex:

```
public class Overloading
{
    public static int sum(int i, int j)
    {
        return i+j;
    }
    public static float sum(float a, float b)
    {
        return a+b;
    }
    public static double sum(double d, double e)
    {
        return d+e;
    }
    public static String sum(char c1, String c2)
    {
        return c1+c2;
    }
    public static String sum(String s1, String s2)
    {
        return s1+s2;
    }
    public static void main(String[] args)
    {
        System.out.println("Sum of two ints = "+sum(10,20));
        System.out.println("Sum of two floats = "+sum(10.1f,20.2));
        System.out.println("Sum of two doubles = "+sum(10.1,20.2));
        System.out.println("Sum of char and string = "+sum('A',"B"));
        System.out.println("Sum of two strings = "+sum("Raghu"," College"));
    }
}
```

Output:

```
Sum of two ints = 30
Sum of two floats = 30.300000381469726
Sum of two doubles = 30.299999999999997
Sum of char and string = AB
Sum of two strings = Raghu College
```

Constructor

- It is a special type of method which is used to initialize the object.
- Every time an object is created using the new () keyword, at least one constructor is called.
- It calls a default constructor if there is no constructor available in the class. In such case, Java compiler provides a default constructor by default.

RULES/PROPERTIES/CHARACTERISTICS OF A CONSTRUCTOR:

1. Constructor name must be the same as its class name
2. A Constructor must have no explicit return type
3. Constructors can be overloaded. So, there can be any number of constructors for a class.
4. Constructors will not be inherited at all.
5. A Java constructor cannot be **abstract**, **static**, **final**, and **synchronized**.
6. Constructors are called automatically whenever an object is created.

Types of Java constructors

There are two types of constructors in Java:

1. Default constructor (no-arg constructor)
2. Parameterized constructor

Default Constructor

A constructor is called "Default Constructor" when it doesn't have any parameter.

Syntax:

```
<class name>()  
{  
  
}
```

Ex:

```
class Sample  
{  
    Sample()  
    {  
        System.out.println("Default Constructor Called");  
    }  
    public static void main(String[] args)  
    {  
        Sample s1 = new Sample();  
        Sample s2 = new Sample();  
    }  
}
```

Output:

```
Default Constructor Called  
Default Constructor Called
```

Purpose of default constructor

The default constructor is used to provide the default values to the object like 0, null, etc., depending on the type.

Ex:

```
class Sample
{
    int i;
    String s;
    Sample()
    {
        System.out.println("Default Constructor Called");
        System.out.println(i+" "+s);
    }
    public static void main(String[] args)
    {
        Sample s1 = new Sample();
    }
}
```

Output:

```
Default Constructor Called
0 null
```

NOTE:

Whenever we create an object only with default constructor, defining the default constructor is optional. If we are not defining default constructor of a class, then JVM will call automatically system defined default constructor (SDDC). If we define, JVM will call user/programmer defined default constructor (UDDC).

Parameterized Constructor

A constructor which has a specific number of parameters is called a parameterized constructor.

Syntax:

```
<class name> <parameters>
{
}
}
```

purpose of parameterized constructor

The parameterized constructor is used to provide different values to distinct objects. However, you can provide the same values also.

Ex:

```
class Constructor
{
    int i;
    String s;
```

```
    Constructor (int a, String name)
    {
        i=a;
        s=name;
    }
    void display()
    {
        System.out.println("Parameterized Constructor Called");
        System.out.println(i+" "+s);
    }
    public static void main(String[] args)
    {
        Constructor s1 = new Constructor (1,"REI");
        s1.display();
    }
}
```

Output:

```
Parameterized Constructor Called
1 REI
```

NOTE:

Whenever we create an object using parameterized constructor, it is mandatory for the JAVA programmer to define parameterized constructor otherwise we will get compile time error.

Constructor Overloading in Java

Overloaded constructor is one in which constructor name is similar but its signature is different. Signature represents number of parameters, type of parameters and order of parameters. Here, at least one thing must be differentiated.

Ex:

class Sample

```
{
    int i;
    String s;
    Sample()
    {
        System.out.println("Default Constructor Called");
        System.out.println(i+" "+s);
    }
    Sample(int i,String name)
    {
        System.out.println("Parameterized Constructor Called");
        System.out.println(i+" "+name);
    }
    public static void main(String[] args)
    {
```



```

        Sample s1 = new Sample();
        Sample s2 = new Sample(1,"REC");
    }
}

```

Output:

Default Constructor Called
 0 null
 Parameterized Constructor Called
 1 REC

Difference between Default Constructor and Parameterized Constructor:

Default Constructor	Parameterized Constructor
Default constructor is useful to initialize all the objects with same data.	Parameterized constructor is useful to Initialize each object with different data.
Default constructor does not have any parameters.	Parameterized constructor will have 1 or more parameters.
When the data is not passed at the time of creating object, default constructor is called.	When the data is passed at the time of creating object, default constructor is called.

NOTE:

Whenever we define/create the objects with respect to both parameterized constructor and default constructor, it is mandatory for the JAVA programmer to define both the constructors.

NOTE:

When we define a class, that class can contain two categories of constructors they are single default constructor and 'n' number of parameterized constructors (overloaded constructors).

Difference between constructor and method in Java

Java Constructor	Java Method
A constructor is used to initialize the state of an object.	A method is used to expose the behaviour of an object.
A constructor must not have a return type.	A method must have a return type.
The constructor is invoked implicitly.	The method is invoked explicitly.
The Java compiler provides a default constructor if you don't have any constructor in a class.	The method is not provided by the compiler in any case.
The constructor name must be same as the class name.	The method name may or may not be same as the class name.

Difference between Constructor Overloading and Method Overloading in Java

Constructor Overloading

1. Writing more than 1 constructor in a class with a unique set of arguments is called as Constructor Overloading
2. All constructors will have the name of the class
3. Overloaded constructor will be executed at the time of instantiating an object
4. An overloaded constructor cannot be static as a constructor relates to the creation of an object
5. An overloaded constructor cannot be final as constructor is not derived by subclasses it won't make sense
6. An overloaded constructor can be private to prevent using it for instantiating from outside of the class

Method Overloading

1. Writing more than one method within a class with a unique set of arguments is called as method overloading
2. All methods must share the same name
3. An overloaded method if not static can only be called after instantiating the object as per the requirement
4. Overloaded method can be static, and it can be accessed without creating an object
5. An overloaded method can be final to prevent subclasses to override the method
6. An overloaded method can be private to prevent access to call that method outside of the class

Garbage Collector:

- In java, garbage means unreferenced objects.
- Garbage Collection is process of reclaiming the runtime unused memory automatically. In other words, it is a way to destroy the unused objects.

Advantages

- It makes java **memory efficient** because garbage collector removes the unreferenced objects from heap memory.
- It is **automatically done** by the garbage collector (a part of JVM) so we don't need to make extra efforts.

An object is unreferenced in the following ways.

- By nulling the reference
- By assigning a reference to another
- By anonymous object etc.

1) By nulling a reference:

```
Employee e=new Employee();  
e=null;
```

2) By assigning a reference to another:

```
Employee e1=new Employee();  
Employee e2=new Employee();  
e1=e2;    //now the first object referred by e1 is available for garbage collection
```

3) By anonymous object:

```
new Employee();
```

In order to call the garbage collector explicitly, we use finalize() method and gc() method.

finalize() method

The finalize() method is invoked each time before the object is garbage collected. This method can be used to perform cleanup processing.

```
protected / public void finalize(){}
```

gc() method

The gc() method is used to invoke the garbage collector to perform cleanup processing. The gc() is found in System and Runtime classes.

```
public static void gc(){}
```

Ex:

```
public class GcDemo {
```

```
    public void finalize()  
    {  
        System.out.println("Object Memory Destroyed");  
    }  
    public static void main(String[] args) {
```

```
        GcDemo gc1 = new GcDemo();  
        gc1 = null;
```

```
        new GcDemo();  
        System.gc();  
    }
```

```
}
```

Output:

```
Object Memory Destroyed  
Object Memory Destroyed
```

Java static keyword:

- The static keyword in Java is used for memory management mainly. i.e., it saves memory.
- The static can be:
 - Variable (also known as a class variable)
 - Method (also known as a class method)
 - Block
 - Nested class
- The static keyword belongs to the class than instance of the class.

Static Variable:

If you declare any variable as static, it is known static variable.

- The static variable can be used to refer the common property of all objects (that is not unique for each object)
e.g., company name of employees, college name of students etc.
- The static variable gets memory only once in class area at the time of class loading.

Ex-1:

```
public class FirstProgram
{
    static int c=10;           // Static Variable Declaration and Initialization
    public static void main(String[] args) {

        //Accessing static variable without class name
        System.out.println(c);

        //Accessing static variable with class name
        System.out.println(FirstProgram.c);

        //Accessing static variable with object
        System.out.println(fp.c);
    }
}
```

Output:

```
10
10
10
```

Ex-2:

```
class Student{  
    int rollno;                //instance variable  
    String name;  
    static String college ="REC";    //static variable  
    //constructor to initialize the variable  
    Student(int r, String n){  
        rollno = r;  
        name = n;  
    }  
    //method to display the values  
    void display (){  
        System.out.println(rollno+" "+name+" "+college);  
    }  
}  
  
public class StaticDemo{  
    public static void main(String args[]){  
        Student s1 = new Student(1,"RAGHU");  
        Student s2 = new Student(2,"RAHUL");  
        //we can change the college of all objects by the single line of code  
        //Student.college="RECA";  
        s1.display();  
        s2.display();  
    }  
}
```

Output:

1 RAGHU REC

2 RAHUL REC

Static Method:

If you apply static keyword with any method, it is known as static method.

- A static method belongs to the class rather than object of a class.
- A static method can be invoked without the need for creating an instance of a class.
- static method can access static data member and can change the value of it.
- We cannot access instance variables(non-static) and instance methods(non-static methods) inside the static method.

- It is invoked by using the classname , objectname, and through the instance method.

Ex-1:

```

class Display
{
    void display()
    {
        System.out.println("Welcome To JAVA");
        display1();    // static method calling
    }
    static void display1()
    {
        System.out.println("Hello World");
    }
}

public class ClassDemo {
    public static void main(String[] args) {
        Display d = new Display(); // Object Creation
        d.display();                // instance method calling

        Display.display1();         // static method calling

        d.display1();               // static method calling
    }
}

```

Output:

```

Welcome To JAVA
Hello World
Hello World
Hello World

```

Ex-2:

```

class Student{
    int rollno;                //instance variable
    String name;
    static String college = "REC"; //static variable

    static void change(){
        college = "RIT";
    }

    //constructor to initialize the variable
    Student(int r, String n){
        rollno = r;
        name = n;
    }

    //method to display the values
    void display (){
        System.out.println(rollno+" "+name+" "+college);
    }
}

```

```

    }
}

//we can change the college of all objects by the single line of code
public class StaticDemo{
    public static void main(String args[]){

        Student.change(); // static method calling

        Student s1 = new Student(1,"RAGHU");
        Student s2 = new Student(2,"RAHUL");

        s1.display();
        s2.display();
    }
}

```

Output:

```

1 RAGHU RIT
2 RAHUL RIT

```

Java static block:

- Is used to initialize the static data member.
- It is executed before the main method at the time of class loading.

Ex:

```

class StaticDemo{
    static{
        System.out.println("static block is invoked");
    }

    public static void main(String args[]){
        System.out.println("Main Method");
    }
}

```

Output:

```

static block is invoked

```

```

Main Method

```

Java static Class:

- A class is declared by using static keyword is called as static class.
- A class can be declared static only if it is a **nested class** i.e., It does not require any reference of the outer class.
- The property of the static class is that it does not allows us to access the non-static members of the outer class.

Ex-1:

```
class OuterClass {
    static class InnerClass{
        void innerClassMethod()
        {
            System.out.println("inner Class Method");
        }
    }
    public static void main(String[] args) {
        InnerClass ic = new InnerClass();
        ic.innerClassMethod();
    }
}
```

Output:
inner Class Method

Ex-2:

```
class OuterClass {
    int a =10;
    static int b=20;
    static class InnerClass{
        void innerClassMethod()
        {
            System.out.println("inner Class Method");
            //System.out.println(a);    // invalid
            System.out.println(b);    // valid
        }
    }
    public static void main(String[] args) {
        InnerClass ic = new InnerClass();
        ic.innerClassMethod();
    }
}
```

Output:
inner Class Method
20

Restrictions for the static method

There are two main restrictions for the static method. They are:

1. The static method cannot use non static data member or call non-static method directly.
2. this and super keywords cannot be used in static context.

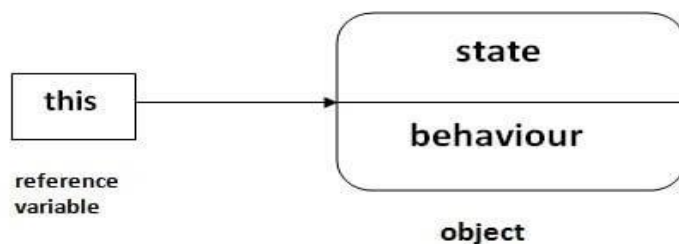
Ex:

```
class A{  
    int a=40;           //non static variable or instance variable  
    public static void main(String args[]){  
        System.out.println(a);  
    }  
}
```

Output: Compile Time Error

this keyword in Java:

In Java, this is a **reference variable** that refers to the current object.



this is an internal or implicit object created by JAVA for the following purposes. They are

- 'this' object is internally pointing to current class object.
- this (): this () is used for calling current class default constructor from current class parameterized constructors.
- this (...): this (...) is used for calling current class parameterized constructor from other category constructors of the same class.

Usage of Java this keyword:

1. this can be used to refer current class instance variable.
2. Whenever the formal parameters and attributes of the class are similar, to differentiate the attributes of the class from formal parameters, the attributes of class must be preceded by 'this'.
3. this can be used to invoke current class method (implicitly)
4. this() can be used to invoke current class constructor.

Rule for 'this':

- Whenever we use either this () or this (...) in the current class constructors, that statements must be used as first statement only.
- The order of the output containing this () or this (...) will be in the reverse order of the input which we gave as inputs.

NOTE:

Whenever we refer the attributes which are similar to formal parameters, the JVM gives first preference to formal parameters whereas whenever we write a keyword this before the variable name of a class then the JVM refers to attributes of the class. these methods are used for calling current class constructors.

NOTE:

- If any method called by an object, then that object is known as source object.
- If we pass an object as a parameter to the method then that object is known as target object.

Ex-1:

In this program, we are printing the reference variable and this, output of both variables are same.

```
class Sample{  
    void m(){  
        System.out.println(this);    //prints same reference ID  
    }  
    public static void main(String args[]){  
        Sample obj=new Sample();  
        System.out.println(obj);      //prints the reference ID  
        obj.m();  
    }  
}
```

Output

```
Sample@379619aa  
Sample@379619aa
```

1. this can be used to refer current class instance variable.

Whenever the instance variable and local variable of the class are similar, to differentiate the instance variable of the class from local variable, the instance variable of class must be preceded by 'this'.

Example:

```
class ThisDemo
{
    int a = 10;
    void display()
    {
        int a = 20;
        System.out.println(this.a);
        System.out.println(a);
    }
    public static void main(String[] args)
    {
        ThisDemo td = new ThisDemo();
        td.display();
    }
}
```

Output:

```
10
20
```

2. Whenever the formal parameters and attributes of the class are similar, to differentiate the attributes of the class from formal parameters, the attributes of class must be preceded by 'this'.

Example:

```
class ThisDemo
{
    int i;
    String s;
    ThisDemo(int i,String s)
    {
        this.i=i;
        this.s=s;
    }
    void print()
    {
        System.out.println(i+" "+s);
    }
    public static void main(String[] args)
    {
        ThisDemo td = new ThisDemo(1,"REI");
        td.print();
    }
}
```

Output:

```
1 REI
```

3. this can be used to invoke current class method (implicitly)

You may invoke the method of the current class by using this keyword. If you don't use this keyword, compiler automatically adds this keyword while invoking the method.

Example:

```
class A{
    void m()
    {
        System.out.println("Instance Method m");
    }
    void n(){
        System.out.println("Instance Method n");
        //m(); //same as this.m()
        this.m();
    }
}
class ThisDemo{
    public static void main(String args[]){
        A a=new A();
        a.n();
    }
}
```

Output:

```
Instance Method m
Instance Method n
```

4. this() can be used to invoke current class constructor.

The this() constructor call can be used to invoke the current class constructor. It is used to reuse the constructor. In other words, it is used for constructor chaining.

Ex-1: Calling default constructor from parameterized constructor

```
class Sample
{
    int i;
    String s;
    Sample()
    {
        System.out.println("Default Constructor Called");
    }
    Sample(int i,String s)
    {
        this();
        this.i=i;
        this.s=s;
    }
    void display()
    {
        System.out.println(i+" "+s);
    }
}
```

```
    }  
    public static void main(String[] args)  
    {  
        Sample s1 = new Sample(1,"REC");  
        s1.display();  
    }  
}
```

Output:

Default Constructor Called

1 REC

Ex-2: Calling parameterized constructor from default constructor

class Sample

```
{  
    int i;  
    String s;  
    Sample()  
    {  
        this(1,"REC");  
        System.out.println("Default Constructor Called");  
    }  
    Sample(int i,String s)  
    {  
        this.i=i;  
        this.s=s;  
    }  
    void display()  
    {  
        System.out.println(i+" "+s);  
    }  
    public static void main(String[] args)  
    {  
        Sample s1 = new Sample();  
        s1.display();  
    }  
}
```

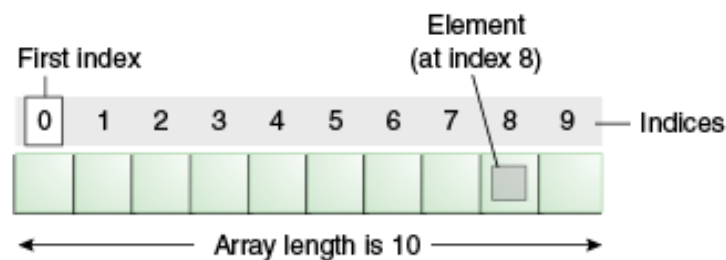
Output:

Default Constructor Called

1 REC

Java Arrays:

- An array is a collection of similar data values with a single name. An array can also be defined as, a special type of variable that holds multiple values of the same data type at a time.
- In java, arrays are objects and they are created dynamically using new operator. Every array in java is organized using index values. The index value of an array starts with '0' and ends with 'size-1'. We use the index value to access individual elements of an array.
- Array in Java is index-based, the first element of the array is stored at the 0th index, 2nd element is stored on 1st index and so on.



Advantages

- Code Optimization: It makes the code optimized, we can retrieve or sort the data efficiently.
- Random access: We can get any data located at an index position.

Disadvantages

Size Limit: We can store only the fixed size of elements in the array. It doesn't grow its size at runtime. To solve this problem, collection framework is used in Java which grows automatically.

Types of Arrays

There are two types of arrays.

- Single Dimensional Array
- Multidimensional Array

Single Dimensional Array:

- In Java, single dimensional arrays are used to store list of values of same datatype.
- In other words, single dimensional arrays are used to store a row of values.
- In single dimensional array, data is stored in linear form.
- Single dimensional arrays are also called as **one-dimensional arrays**, **Linear Arrays** or simply **1-D Arrays**.

Syntax to Declare an Array in Java

```
dataType[] arr; (or)  
dataType []arr; (or)  
dataType arr[];
```

Instantiation of an Array in Java

arrayRefVar=new datatype[size];

Ex-1:

```
class Testarray{
    public static void main(String args[]){
        int a[]=new int[5]; //declaration and instantiation
        a[0]=10;           //initialization
        a[1]=20;
        a[2]=70;
        a[3]=40;
        a[4]=50;
        //traversing array
        for(int i=0;i<a.length;i++)//length is the property of array
            System.out.println(a[i]);
    }
}
```

Output:

```
10
20
70
40
50
```

Ex-2:

```
class Testarray1{
    public static void main(String args[]){
        int a[]={33,3,4,5}; //declaration, instantiation and initialization
        //printing array
        for(int i=0;i<a.length;i++) //length is the property of array
            System.out.println(a[i]);
    }
}
```

Output:

```
33
3
4
5
```

For-each Loop for Java Array

```
for(data_type variable:array){  
    //body of the loop  
}
```

Ex:

```
class Testarray1{  
    public static void main(String args[]){  
        int arr[]={33,3,4,5};  
        //printing array using for-each loop  
        for(int i:arr)  
            System.out.println(i);  
    }  
}
```

Output:

```
33  
3  
4  
5
```

Multidimensional Array:

An array of arrays is called as multi-dimensional array. In simple words, an array created with more than one dimension (size) is called as multi-dimensional array. Multi-dimensional array can be of **two-dimensional array** or **three-dimensional array** or **four-dimensional array** or more... Most popular and commonly used multi-dimensional array is **two-dimensional array**. The 2-D arrays are used to store data in the form of table. We also use 2-D arrays to create mathematical **matrices**.

Syntax to Declare Multidimensional Array:

```
dataType[][] arrayRefVar; (or)  
dataType [][]arrayRefVar; (or)  
dataType arrayRefVar[][]; (or)  
dataType []arrayRefVar[];
```

Example to instantiate Multidimensional Array:

```
int[][] arr=new int[3][3];    //3 row and 3 column
```

Example to initialize Multidimensional Array:

```
arr[0][0]=1;  
arr[0][1]=2;  
arr[0][2]=3;
```



```
arr[1][0]=4;  
arr[1][1]=5;  
arr[1][2]=6;  
arr[2][0]=7;  
arr[2][1]=8;  
arr[2][2]=9;
```

Example:

```
class Testarray3{  
    public static void main(String args[]){  
        //declaring and initializing 2D array  
        int arr[][]={{1,2,3},{2,4,5},{4,4,5}};  
        //printing 2D array  
        for(int i=0;i<3;i++){  
            for(int j=0;j<3;j++){  
                System.out.print(arr[i][j]+" ");  
            }  
            System.out.println();  
        }  
    }  
}
```

Output:

```
1 2 3  
2 4 5  
4 4 5
```

Java Command Line Arguments

- The java command-line argument is an argument i.e. passed at the time of running the java program.
- The arguments passed from the console can be received in the java program and it can be used as an input.
- So, it provides a convenient way to check the behaviour of the program for the different values. You can pass N (1,2,3 and so on) numbers of arguments from the command prompt.

Simple example of command-line argument in java

In this example, we are receiving only one argument and printing it. To run this java program, you must pass at least one argument from the command prompt.

```
class CommandLineExample{  
    public static void main(String args[]){  
        System.out.println("Your first argument is: "+args[0]);  
    }  
}
```

Output:

```
E:\>javac CommandLineExample.java  
E:\>java CommandLineExample RECCSE  
Your first argument is: RECCSE
```

Example of command-line argument that prints all the values

In this example, we are printing all the arguments passed from the command-line.

For this purpose, we have traversed the array using for loop.

```
class A{  
    public static void main(String args[]){  
        for(int i=0;i<args.length;i++)  
            System.out.println(args[i]);  
    }  
}
```

Output:

```
E:\>javac A.java  
E:\>java A 1 2 3 Rec Cse  
1  
2  
3  
Rec  
Cse
```

Nested Class / Inner class:

- Java nested class or inner class is a class that is declared inside other class is known as nested class.

Advantage of Java inner classes

- it can access all the members (attributes and methods) of the outer class, including private.
- Nested classes are used to develop more readable and maintainable code.
- **Code Optimization:** It requires less code to write.

Types of Nested classes

1. non-static nested class (inner class)

- Member inner class (Regular inner class)
- Local inner class (Method local inner class)
- Anonymous inner class

2. static nested class (Static inner class)

Member inner class (Regular inner class)

A non-static class that is created inside a class but outside a main method is called **member inner class**. It is also known as a **regular inner class**. It can be declared with access modifiers like public, default, private, and protected.

Ex-1:

```
class OuterClass {
    void outerClassMethod()
    {
        System.out.println("Outer Class Method");
        InnerClass i = new InnerClass();
        i.innerClassMethod();
    }
    class InnerClass
    {
        void innerClassMethod()
        {
            System.out.println("inner Class Method");
        }
    }
    public static void main(String[] args) {
        OuterClass oc = new OuterClass();
        oc.outerClassMethod();

        //oc.innerClassMethod(); // invalid

        //InnerClass ic = new InnerClass(); //invalid

        InnerClass ic = oc.new InnerClass(); //valid
        ic.innerClassMethod();

        OuterClass.InnerClass ic1 = oc.new InnerClass(); //valid
        ic1.innerClassMethod();

        OuterClass.InnerClass ic2 = new OuterClass().new InnerClass(); //valid
        ic2.innerClassMethod();
    }
}
```

Output:

```
Outer Class Method
inner Class Method
inner Class Method
inner Class Method
inner Class Method
```

Ex-2

```

class OuterClass {
    private int a=10;
    void outerClassMethod()
    {
        System.out.println("Outer Class Method");
    }
    class InnerClass // Inner Class
    {
        void innerClassMethod()
        {
            System.out.println(a); // Outer Class variable
            outerClassMethod(); // Outer Class Method
            System.out.println("inner Class Method");
        }
    }
    public static void main(String[] args) {
        OuterClass oc = new OuterClass();
        oc.outerClassMethod();

        InnerClass ic = oc.new InnerClass(); //valid
        ic.innerClassMethod();
    }
}

```

Output:

```

Outer Class Method
10
Outer Class Method
inner Class Method

```

Local inner class (Method local inner class):

- Local Inner Classes are the inner classes that are defined inside a method or block.
- Local Inner classes are not a member of any enclosing classes. They belong to the block they are defined within, due to which local inner classes cannot have any access modifiers associated with them.
- If you want to invoke the methods of the local inner class, you must instantiate this class inside the method.

Ex:

```

class OuterClass {
    void outerClassMethod()
    {
        class InnerClass
        {
            void innerClassMethod()
            {

```

```

        System.out.println("inner Class Method");
    }
}
InnerClass ic = new InnerClass();
ic.innerClassMethod();
}

public static void main(String[] args) {
    OuterClass oc = new OuterClass();
    oc.outerClassMethod();
}
}

```

Output:

inner Class Method

Anonymous inner class

A class that has no name is known as an anonymous inner class in Java.

Ex:

```

class X {
    void display()
    {
        System.out.println("REC");
    }
}
class Y {
    public static void main(String[] args) {
        X x = new X();
        x.display();

        X x1 = new X()           // Anonymous inner class
        {
            void display()
            {
                System.out.println("CSE");
            }
        };
        x1.display();
    }
}

```

Output:

REC
CSE

Static Inner Class

- A class is declared by using static keyword is called as static class.
- A class can be declared static only if it is a **nested class** i.e., It does not require any reference of the outer class.
- The property of the static class is that it does not allows us to access the non-static members of the outer class.

Ex:

```
class OuterClass {  
    static class InnerClass  
    {  
        void innerClassMethod()  
        {  
            System.out.println("inner Class Method");  
        }  
    }  
    public static void main(String[] args) {  
        InnerClass ic = new InnerClass();  
        ic.innerClassMethod();  
    }  
}
```

Output:

inner Class Method